

Fuente: Asignatura de Programación (Tema 14)

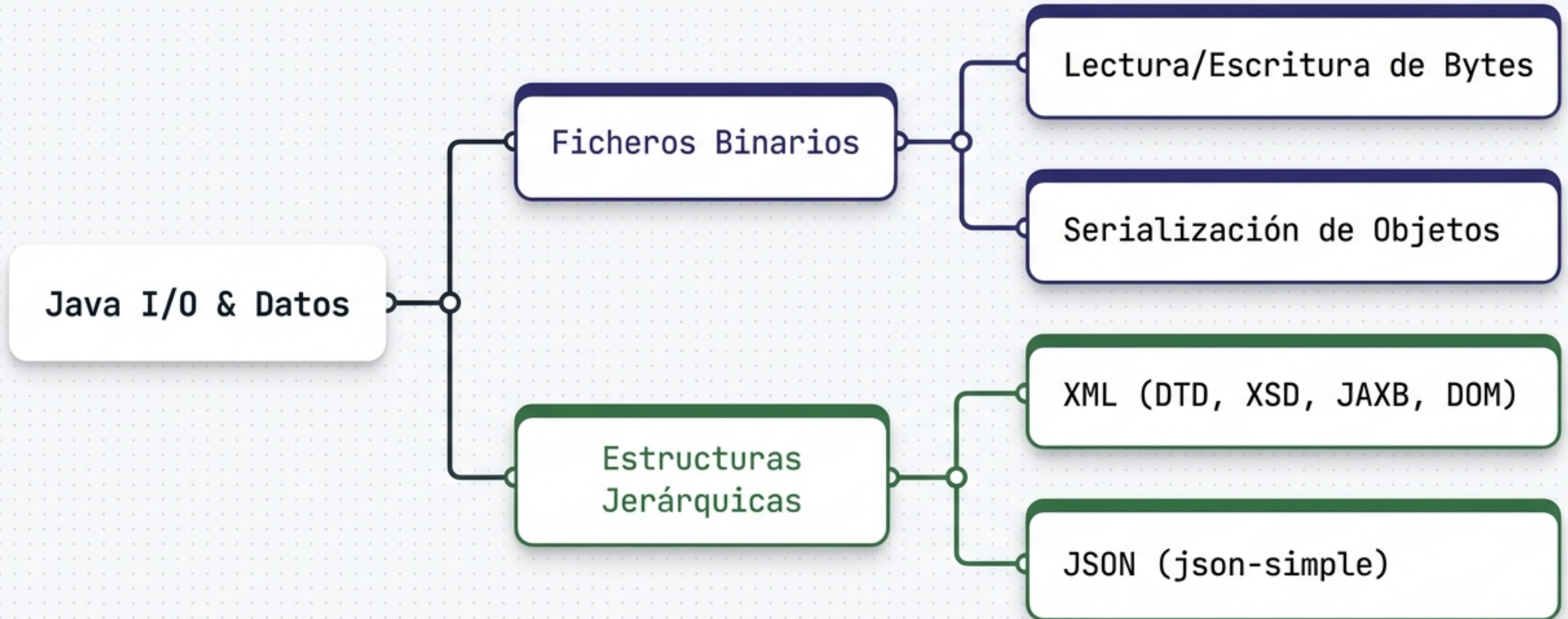
Gestión de Ficheros Binarios y Estructuras Jerárquicas

Persistencia de Datos en Java:
De Bytes a Objetos y Nodos



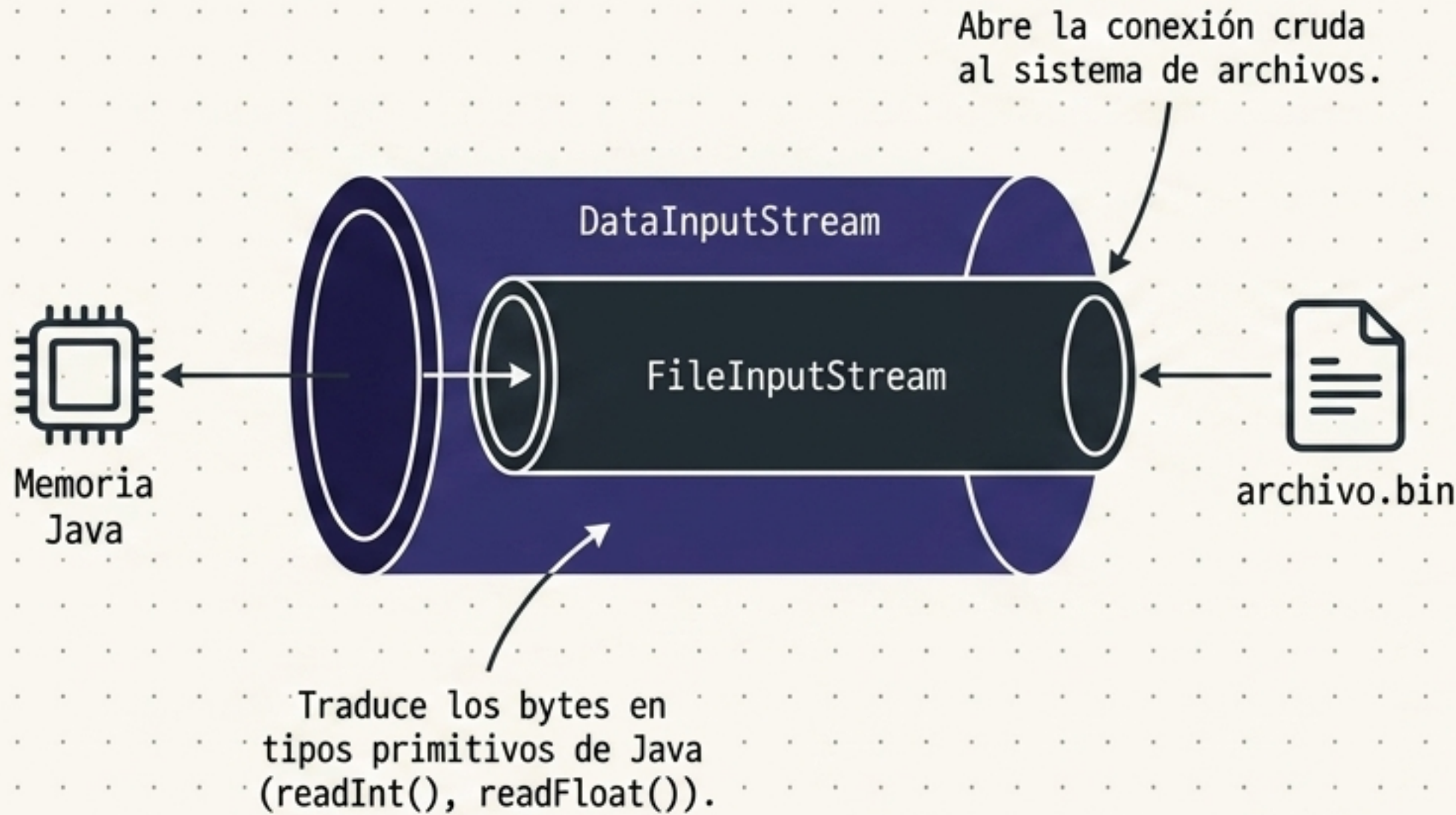
El Ecosistema de Persistencia en Java

La persistencia de datos requiere elegir el nivel correcto de abstracción: desde la manipulación de bytes crudos a nivel de máquina, hasta la exportación de documentos legibles e interoperables.



Lectura de Bytes: Envolviendo Flujos de Datos

Java utiliza un patrón "Wrapper". El flujo base lee bytes, pero envolviéndolo en un `DataInputStream` obtenemos herramientas para leer datos primitivos directamente.



```
try {
    FileInputStream fileInputStream = new
    FileInputStream(filePath);
    DataInputStream dataInputStream = new
    DataInputStream(fileInputStream);

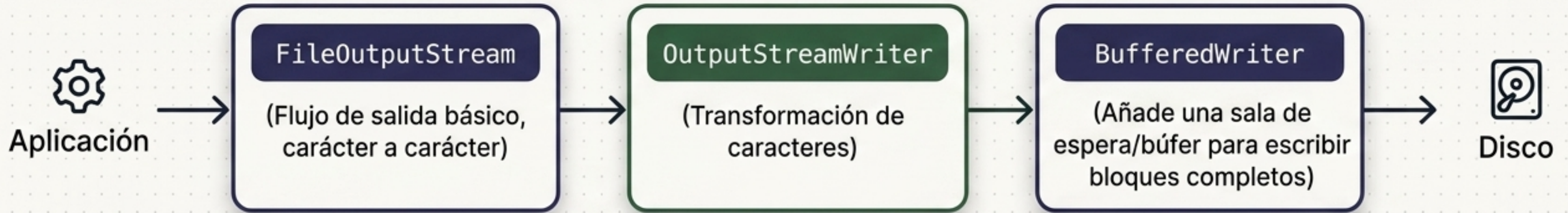
    // Leer datos del archivo binario
    int intValue = dataInputStream.readInt();
    float floatValue = dataInputStream.readFloat();

    // Imprimir los valores leídos
    System.out.println("Valor entero leído: " +
    intValue);
    System.out.println("Valor flotante leído: " +
    floatValue);

    // Cerrar flujos después de su uso
    dataInputStream.close();
    fileInputStream.close();
} catch (IOException e) {
    e.printStackTrace();
}
```


Escritura de Bytes y Eficiencia con Búferes

Al escribir, las clases de búfer evitan saturar el disco agrupando las escrituras.
Recuerda: el texto requiere conversión explícita a bytes y codificación correcta.

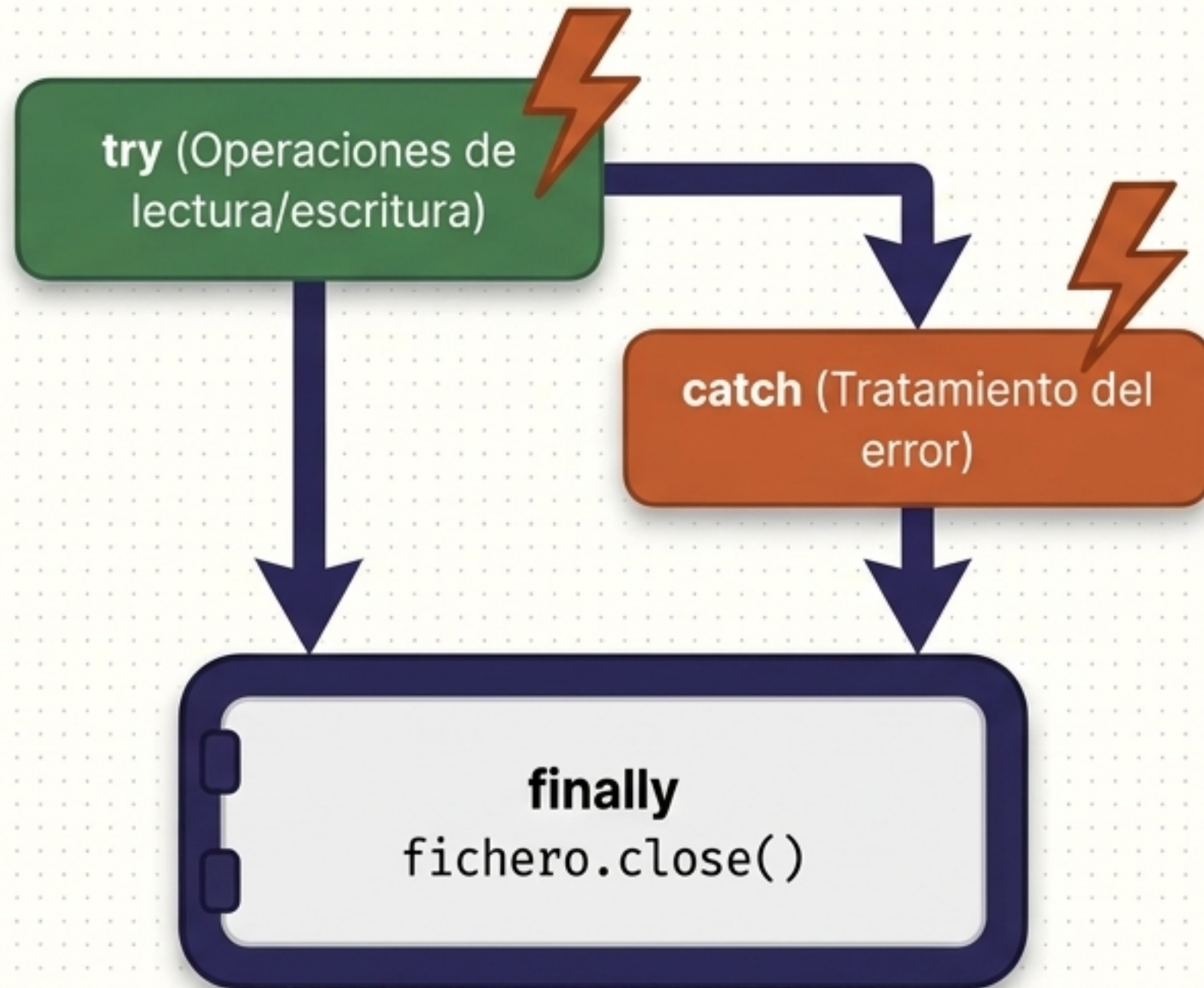


ADVERTENCIA:
`StandardCharsets.UTF_8`
`.write()`

```
try {  
    // Escribir datos en un archivo utilizando UTF-8  
    FileOutputStream fileOutputStream = new FileOutputStream(filePath);  
    OutputStreamWriter outputStreamWriter =  
        new OutputStreamWriter(fileOutputStream, StandardCharsets.UTF_8);  
    BufferedWriter bufferedWriter = new BufferedWriter(outputStreamWriter);  
    bufferedWriter.write("¡Hola mundo!");  
    bufferedWriter.close();  
  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

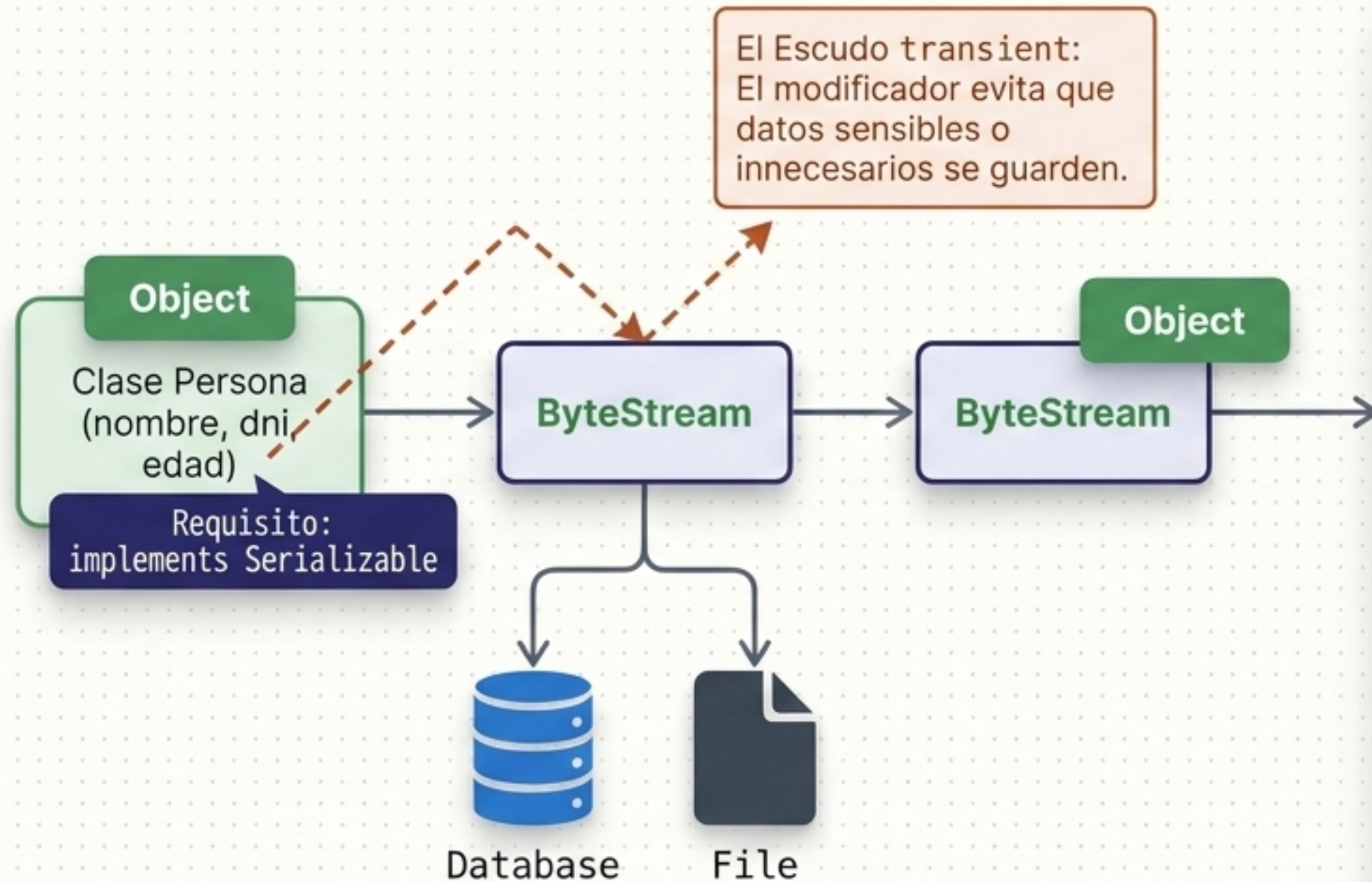

Seguridad de Recursos: El Escudo **finally**

Si ocurre un error en el try o en el catch, las órdenes de cierre pueden saltarse, dejando ficheros abiertos y causando corrupción. El bloque **finally** garantiza la ejecución del cierre sin importar los errores.



```
1  try {  
2    // Operaciones que pueden fallar  
3    // ...  
4  } catch (IOException e) {  
5    System.out.println("Error leyendo el  
6    fichero: " + e.toString());  
7  } finally {  
8    // Este bloque se ejecutará siempre,  
9    // haya excepción o no.  
10   // Aquí es dónde se debe cerrar el  
11   // archivo.  
12   fichero.close();  
13 }
```


Serialización: Guardando Objetos Completos



```
import java.io.Serializable;

/**
 * Esta clase representa una persona
 * @author Francisco Jiménez Moral
 */
public class Persona implements Serializable {

    private String nombre;
    private String dni;
    private transient int edad; // El modificador transient

    // Constructor
    public Persona(String nombre, String dni, int edad) {
        this.nombre = nombre;
        this.dni = dni;
        this.edad = edad;
    }
}
```


Control de Versiones: serialVersionUID

Si no defines un ID manual, el compilador lo generará dinámicamente. Cualquier cambio en la clase romperá la compatibilidad con los objetos previamente guardados.

Objeto guardado en
el archivo .dat
(Versión 1L)



Clase actual en
el IDE



ID Dinámico

La clase se modifica. El compilador cambia el ID automáticamente. Las piezas no encajan -> InvalidClassException.



ID Manual

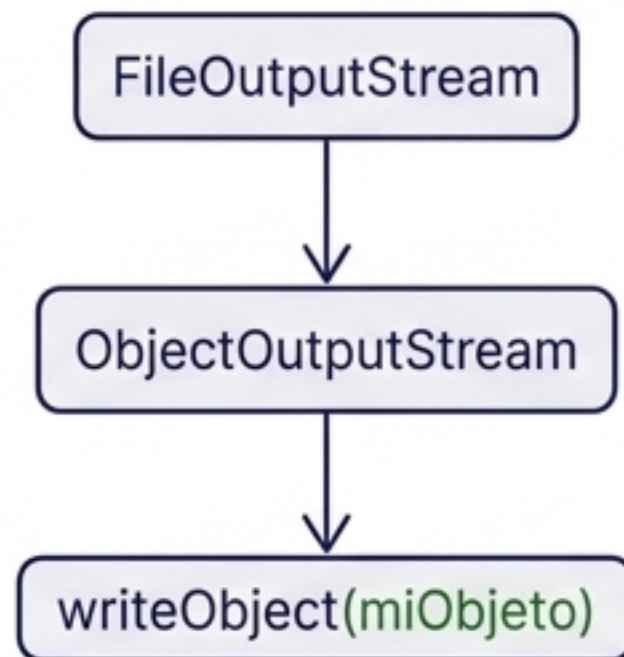
`private static final long serialVersionUID = 1L;`
Las piezas encajan perfectamente aunque haya cambios menores.

```
import java.io.Serializable;
/**
 * Esta clase representa una persona
 * @author Francisco Jiménez Moral
 */
public class Persona implements Serializable {
    private static final long serialVersionUID = 1L;
    private String nombre;
    private String dni;
    private transient int edad;
    // Constructor
    public Persona(String nombre, String dni, int edad) { this.nombre = nombre; this.dni = dni; this.edad = edad; }
}
```


Arquitectura de Lectura y Escritura de Objetos

El flujo de escritura guarda los objetos en formato binario. Al recuperar, Java devuelve un tipo genérico Object, requiriendo un casting explícito al tipo original.

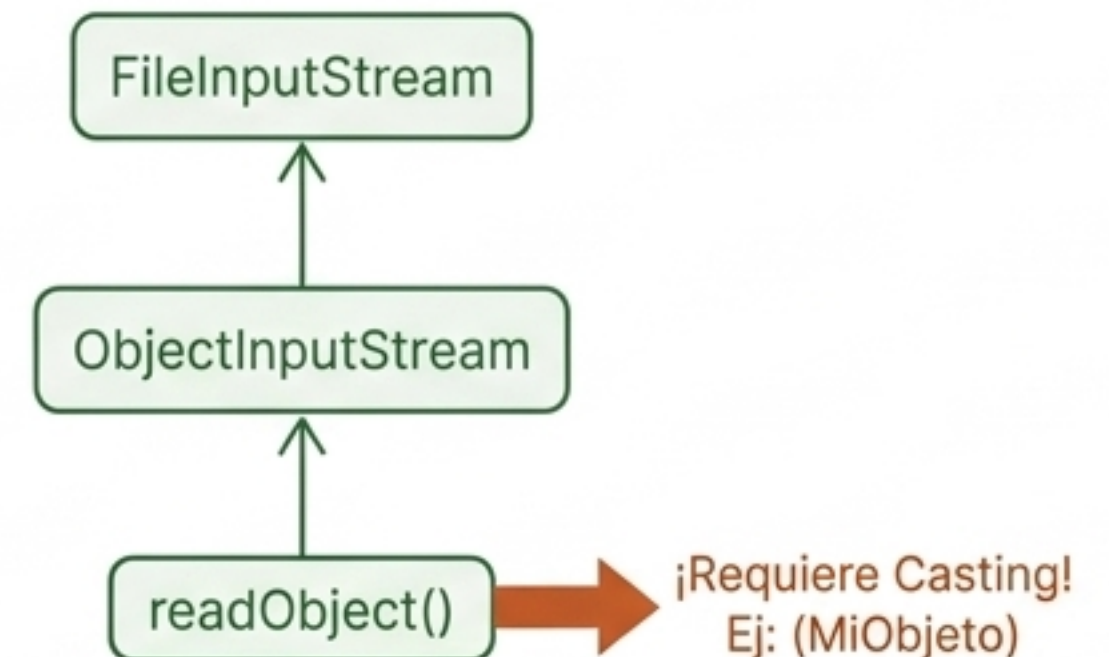
Escritura (Guardar)



```
try (FileOutputStream fileOutputStream = new
    FileOutputStream("objeto.dat");
    ObjectOutputStream objectOutputStream = new
    ObjectOutputStream(fileOutputStream)) {

    // Escribir el objeto en el archivo
    objectOutputStream.writeObject(objetoAEscribir);
} catch (IOException e) { ... }
```

Lectura (Recuperar)

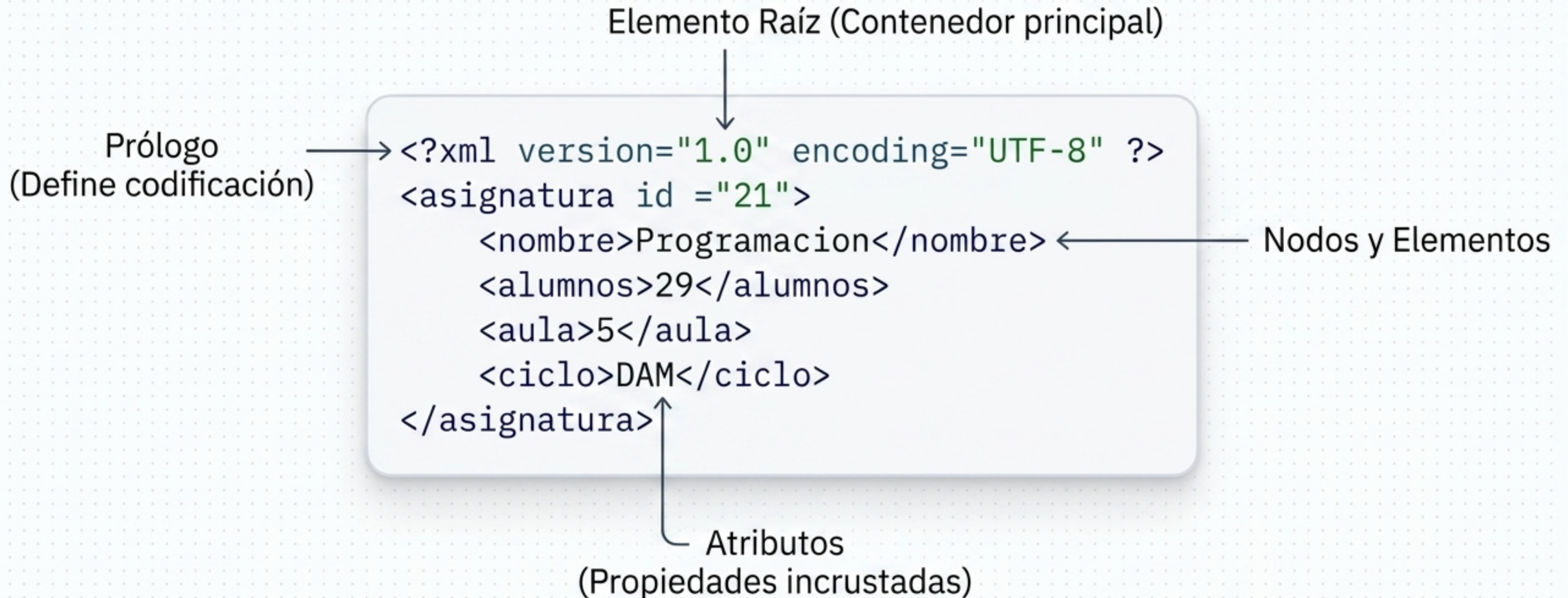


```
try (FileInputStream fileInputStream = new
    FileInputStream("objeto.dat");
    ObjectInputStream objectInputStream = new
    ObjectInputStream(fileInputStream)) {

    // Leer el objeto y hacer casting
    Object objetoLeido = objectInputStream.readObject();
    MiObjeto miObjeto = (MiObjeto) objetoLeido; // ¡Casting explícito!
} catch (IOException | ClassNotFoundException e) { ... }
```


Estructuras Jerárquicas: Anatomía del XML

El XML es un estándar legible por humanos y máquinas, ideal para intercambiar datos estructurados. Se amplía fácilmente añadiendo nuevas etiquetas.



Guardianes de la Estructura: DTD vs. XSD

DTD (Definición de Tipo de Documento)



- Formato más antiguo y básico.
- Define qué elementos están permitidos y su orden (anidamiento).

```
<!ELEMENT biblioteca (libro*)>
<!ELEMENT libro (ISBN, titulo, autor, editorial?)>
<!ELEMENT ISBN (#PCDATA)>
<!ELEMENT titulo (#PCDATA)>
<!ELEMENT autor (#PCDATA)>
<!ELEMENT editorial (#PCDATA)>
```

XSD (XML Schema Definition)



- Formato moderno escrito en propio XML.
- Mayor precisión: define tipos de datos (xsd:string, xsd:double) y rangos.
- Usa Espacios de Nombres (Namespaces).

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Libro">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="Título" type="xsd:string"/>
        <xsd:element name="Autores" type="xsd:string" maxOccurs="10"/>
        <xsd:element name="Editorial" type="xsd:string"/>
      </xsd:sequence>
      <xsd:attribute name="precio" type="xsd:double"/>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```


Vinculación Java-XML: La Magia de JAXB

JAXB (Java Architecture for XML Binding) automatiza la conversión entre clases Java y XML mediante anotaciones, eliminando la necesidad de escribir rutinas de parseo manuales.

Java con Anotaciones JAXB

```
@XmlRootElement(name = "asignatura")
@XmlAccessorType(XmlAccessType.FIELD)
public class Asignatura {
    @XmlAttribute(name = "id")
    private Integer identificación;
    @XmlElement(name = "nombre")
    private String nombreAsignatura;
    @XmlElement
    private String alumnos;
    @XmlElement
    private String aulaReferencia;
    @XmlElement
    private String ciclo;

    // Constructor y getters/setters omitidos
}
```

Salida XML Hipotética

Mapea a raíz

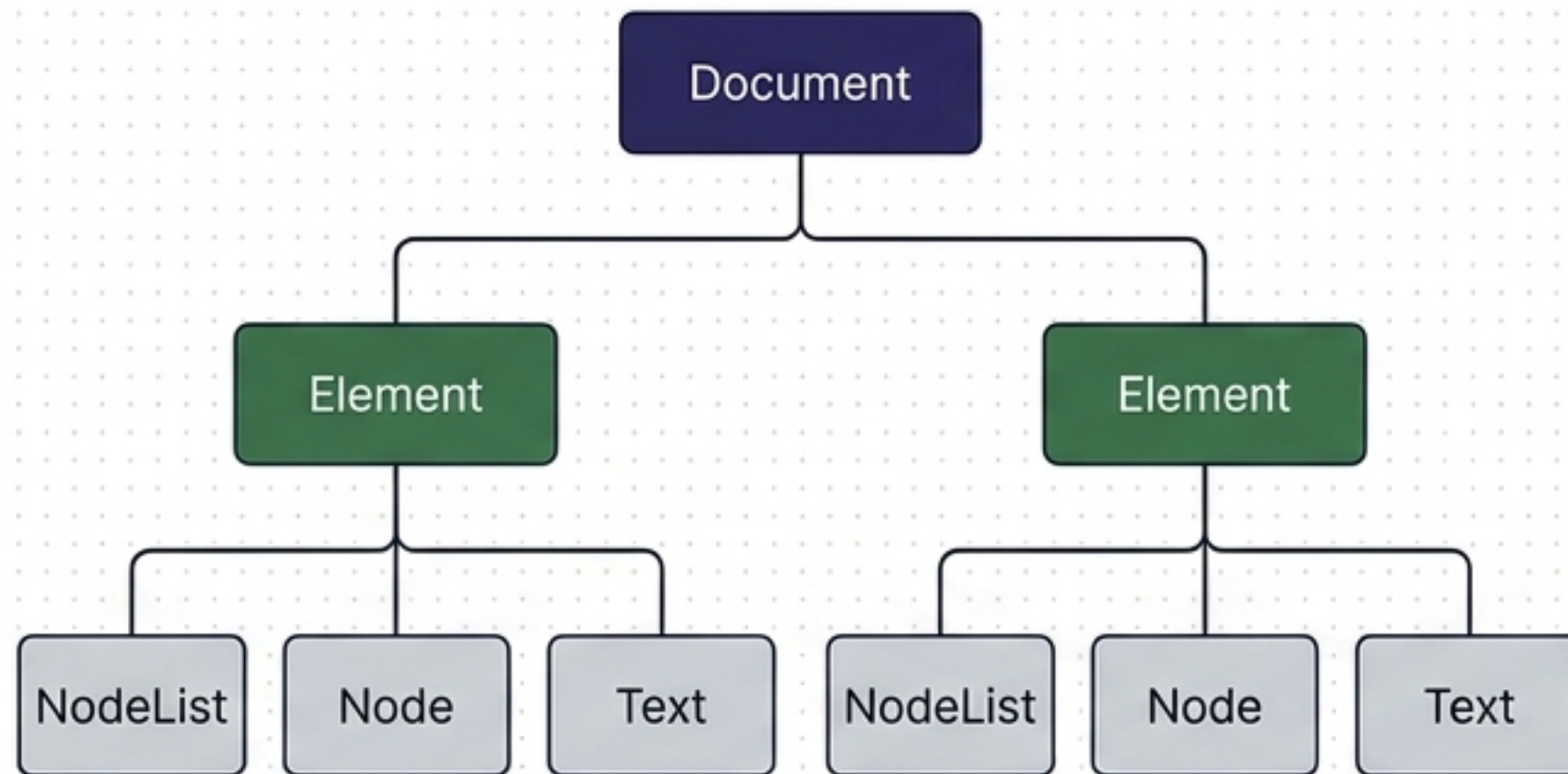
Mapea a atributo

Mapea a elemento

```
<asignatura id="X">
  <nombre>Programación</nombre>
  <alumnos>30</alumnos>
  <aulaReferencia>Aula 5</aulaReferencia>
  <ciclo>DAM</ciclo>
</asignatura>
```


Manipulación en Memoria: Biblioteca DOM

Integrada en la JDK (javax.xml.parsers), DOM carga todo el XML en memoria creando un árbol de nodos. Permite una navegación bidireccional extremadamente rápida, aunque consume más RAM.



```
//-----Obtener un Analizador de XML (Parser)-----  
import javax.xml.parsers.DocumentBuilderFactory;  
import javax.xml.parsers.DocumentBuilder;  
import org.w3c.dom.Document;  
  
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();  
DocumentBuilder builder = factory.newDocumentBuilder();  
  
//-----Analizar un Documento XML-----  
Document document = builder.parse(new File("documento.xml"));  
  
//-----Trabajar con el Documento DOM-----  
// Obtener el elemento raíz  
Element rootElement = document.getDocumentElement();  
  
// Obtener elementos hijos  
NodeList nodeList = rootElement.getChildNodes();  
  
// Recorrer los elementos hijos  
for (int i = 0; i < nodeList.getLength(); i++) {  
    Node node = nodeList.item(i);  
    if (node.getNodeType() == Node.ELEMENT_NODE) {  
        Element element = (Element) node;  
        // Trabajar con los elementos...  
    }  
}  
  
//-----Modificar el Documento DOM-----  
  
// Crear un nuevo elemento  
Element newElement = document.createElement("nuevoElemento");  
newElement.setAttribute("atributo", "valor");  
  
// Añadir el nuevo elemento como hijo del elemento raíz  
rootElement.appendChild(newElement);  
  
//-----Escribir el Documento DOM Modificado a un Archivo XML-----  
TransformerFactory transformerFactory = TransformerFactory.newInstance();  
Transformer transformer = transformerFactory.newTransformer();  
DOMSource source = new DOMSource(document);  
StreamResult result = new StreamResult(new File("documento_modificado.xml"));  
transformer.transform(source, result);
```


JSON: La Alternativa Ligera

JSON es menos verboso que el XML. Usamos librerías ligeras como JSON.simple para intercambios rápidos de datos clave-valor.

Integración



Libraries



json-simple-1.1.1.jar



JDK 1.8 (Default)

Implementación

Parsear texto a JSON

```
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;

public class Main {
    public static void main(String[] args) {
        JSONParser parser = new JSONParser();
        String jsonString = "{\\\"nombre\\\": \\\"Juan\\\", \\\"edad\\\": 30}";

        try {
            JSONObject jsonObject = (JSONObject) parser.parse(jsonString);
            String nombre = (String) jsonObject.get("nombre");
            long edad = (long) jsonObject.get("edad");

            System.out.println("Nombre: " + nombre);
            System.out.println("Edad: " + edad);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Crear JSON desde Java

```
import org.json.simple.JSONObject;

public class Main {
    public static void main(String[] args) {
        JSONObject jsonObject = new JSONObject();
        jsonObject.put("nombre", "Maria");
        jsonObject.put("edad", 25);

        String jsonString = jsonObject.toJSONString();
        System.out.println(jsonString);
    }
}
```


Matriz de Decisión Arquitectónica

Caso Práctico: Sistema de pedidos de un bar. La elección depende del volumen y la necesidad de interoperabilidad.

Ficheros de Texto

Ideal para logs simples y configuración.
Legible por humanos.
Rendimiento lento.

Ficheros Binarios

Ideal para flujos continuos y eficientes dato por dato.
Ejemplo: Registrar cada ticket individual en tiempo real.

Serialización de Objetos

Ideal para guardar estructuras complejas masivamente.
masivamente. Ilegible. Rápido de recuperar en Java.
Ejemplo: Guardar el estado de todas las mesas al cerrar.

XML / JSON

Ideal para interoperabilidad con otros sistemas (bases de datos externas, APIs web).

El Plano Arquitectónico de la Persistencia

Desde la manipulación microscópica de un byte hasta la orquestación de estructuras jerárquicas, Java ofrece un ecosistema escalable.

